CRDTs

data types that merge without conflicts

Watch the 4-min video

In one line: CRDTs (conflict-free replicated data types) are counters, sets, maps and text whose replicas can be updated independently and merged in any order with the same result, because the merge is commutative, associative and idempotent; that gives strong eventual consistency without coordination.

1 Merging without a referee

Two people edit the same shared document offline. When they reconnect, their changes must merge, with no central server deciding and no conflicts to fix by hand. **CRDTs** make that possible by designing the data type so that any two replicas can always be merged, and every replica that has seen the same updates ends up in the same state. This property is called **strong eventual consistency**.

2 The simplest one: a grow-only counter

A **G-counter** keeps a vector with one entry per node. To increment, a node bumps only its own entry. The value is the sum of all entries. To merge two replicas, take the **maximum** of each entry. No increment is ever lost or counted twice, whatever order the merges happen in.

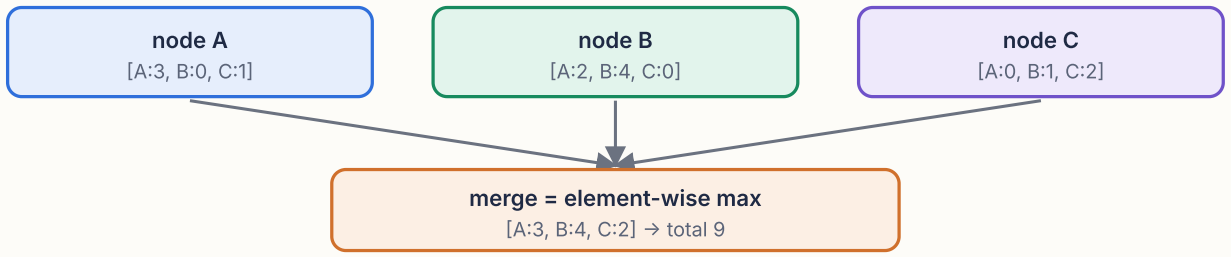


Figure 1. Merging G-counters by element-wise max. Each node only ever increases its own slot.

3 Why it works



Figure 2. If merge has these three properties, replicas converge no matter how messages are delayed, reordered or duplicated.

Mathematically the states form a **join-semilattice** and merge is the join. In practice: design the state so it only 'grows' in a well-defined direction and the merge picks the upper bound. There are two styles: **state-based** CRDTs send whole states (or deltas) and merge them; **operation-based** CRDTs send operations that must commute.

G-Counter	grow-only counter (views)
PN-Counter	two G-counters: increments − decrements (likes)
G-Set / 2P-Set	add-only set / add then remove forever
OR-Set	add and remove, add wins over concurrent remove (carts)
LWW-Register	single value, latest timestamp wins
Sequence (RGA, YATA)	ordered text for collaborative editing

Figure 3. Common CRDTs and what they're used for.

WORKED EXAMPLE: A LIKE COUNTER AND A SHOPPING CART

Likes use a **PN-counter**: one G-counter for likes and one for unlikes, value = P − N. A user in the EU and one in the US like at the same moment; each region bumps its own slot, and after merging both regions show +2. A cart uses an **OR-set** (observed-remove set): each add is tagged with a unique ID, and a remove deletes only the tags it has seen. If one device removes the charger while another, concurrently, adds it again, the new add has an unseen tag, so the charger stays: add wins. Redis Enterprise active-active, Riak and Azure Cosmos DB offer CRDT types; Automerge and Yjs bring them to collaborative apps.

4 Limits

- Not every rule fits: 'balance must never go below zero' or 'username must be unique' need coordination.
- **Metadata grows:** tombstones and per-node entries need garbage collection.
- The merged result is always well defined, but not always what a user expects; design the semantics deliberately.

COMMON MISTAKES

- A counter that stores just a number and merges by max or last-write-wins: concurrent increments are lost.
- Using CRDTs for invariants that need a global check.
- Never compacting tombstones.

5 Key terms

CRDT: conflict-free replicated data type

Strong eventual consistency: same updates seen → same state

State-based CRDT: replicas exchange and merge states

Operation-based CRDT: replicas exchange commuting operations

OR-Set: set where a concurrent add wins over remove

Tombstone: marker for a removed element

6 Interview questions

Q1. What is a CRDT? BASIC

A replicated data type whose replicas can be updated independently and merged in any order to the same state, because the merge is commutative, associative and idempotent.

Q2. How does a G-counter work? BASIC

Each node increments its own entry in a vector; the value is the sum; merging takes the element-wise maximum.

Q3. Design a global like counter across regions without coordination. SCENARIO

Use a PN-counter CRDT replicated per region, update locally, merge asynchronously; show the local value with read-your-writes for the liker.

Q4. Could you use a CRDT for a bank balance with no overdraft? SCENARIO

Not alone: the no-overdraft rule needs a global check. You could use escrow (pre-split the allowed spend between regions) or route withdrawals through a single leader.

Q5. Why must the merge be idempotent? TRICKY

Networks duplicate and retry messages; merging the same state twice must not change the result, or counts would grow on every retry.

Q6. How do collaborative editors like Google Docs or Figma relate to CRDTs? TRICKY

Google Docs historically used operational transformation with a central server; many newer tools, and libraries like Yjs and Automerge, use sequence CRDTs so peers can merge without a server.

Next: SD-H-020 · Idempotent Processing backendroadmap.com